



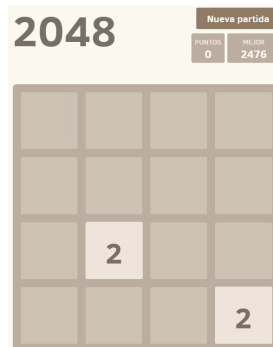
Session 1. Game2048.

1 Instructions

- This exercise must be implemented and submitted **as latest 24 hours before** your next laboratory class.
- Rename the project as `surname1_surname2_name_lab01_game2048` in lowercase and without accents. Then, export the project to an Archive file named `surname1_surname2_name_lab01` and upload it to the corresponding task in Campus Virtual.

2 Introduction

2048 is an online game created in 2014 by Gabriele Cirulli (with 19 years old), whose objective is to move the numbers of the board to combine them until getting in a cell of the board the number 2048, 4096, ... You can try it in: <https://play2048.co/>



2048 is played on a gray 4×4 board with cells that slide smoothly when a player moves them using one of the four arrow keys - Up, Down, Left or Right. On every turn, a new cell with a value of either 2 or 4 randomly appears on an empty spot of the board.

After one of the keys is pressed, the **cells slide as far as possible in the chosen direction until they are stopped either by another cell or by the edge of the board.**

If **two cells with the same number collide while moving**, they **merge** into a cell with this number doubled.

You can't merge an already merged tile in the same turn.

If there are **more than 2 cells** in the same row (column) that can merge, **the farthest** (closest to the edge in the direction of the movement) pair will be merged first.

The game stops when the whole board fills with cells or a single cell value 2048 is achieve.

2.1 Sample output

- Initial screen (2 at a random position)

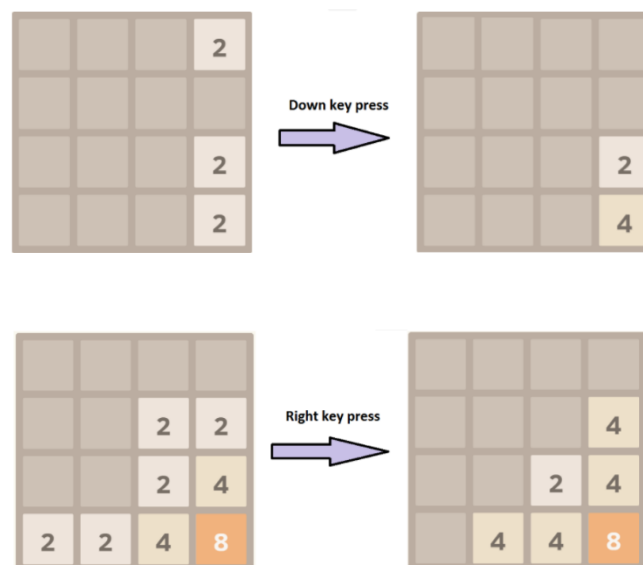
```
2048
0 0 0
0 0 2
0 0 0
```

Choose one direction [r R]/[l L]/[u U]/[d D]:



- User must enter [r R] Right, [l L] Left, [u U] Up [d D] Down to simulate arrows
- Choose one direction [r R]/[l L]/[u U]/[d D]: r next board would be:
2 0 0
0 0 2
0 0 0
- Choose one direction [r R]/[l L]/[u U]/[d D]: L
2 0 0
2 0 0
0 0 2
- Choose one direction [r R]/[l L]/[u U]/[d D]: U
4 0 2
0 0 0
2 0 0

2.2 Some complex examples



2.3 Example with two moves

Consider this sequence of moves: L then R

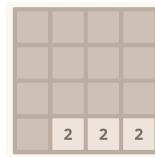


Figure 1: Initial state



Figure 2: After moving left



Figure 3: After moving right

3 Implementing the game

Your task is to implement a simplified version of the original game starting on the skeleton provided. Carefully read the following specifications.

3.1 Specifications

- To maintain the state of the game you must use a **squared matrix** which corresponds to the 2048 game board. `board[0][0]` corresponds to the upper left cell of the board.
- Each cell of the board must contain a **power of 2** integer greater or equal zero, where zero means **the cell is empty**. By default, every cell contains zero.
- By default, the **size** of the board is 3, but any size from **3 to 5** is allowed.
- The goal of the game is to merge cells in order to create a cell containing 2048 (2^{11}).
- Each turn, the player can run a movement (up, down, right or left), and each movement will probably result in a change to the board.
 - Change can be from cell sliding, and/or
 - merging same number tiles by sliding them together along a row/column line.
 - Each time a movement is done, **one new number 2 will randomly be added** to the board, on an empty cell.
- Player chooses a direction along a line (row/column)
 - All cells slide and merge in that direction, if possible.
 - **From the front to the back**, contiguous identical pairs of cells merge into a new cell with their sum.

- Merged cells cannot merge again in the same movement
- All cells slide as far as possible in the direction of the motion

3.2 Class Game2048

This class must hold the necessary constants (the minimum size of the board is 3 and maximum is 5, default is 3), and the board as an array of integers, with the following methods.

3.2.1 Constructors

Three constructors must be implemented:

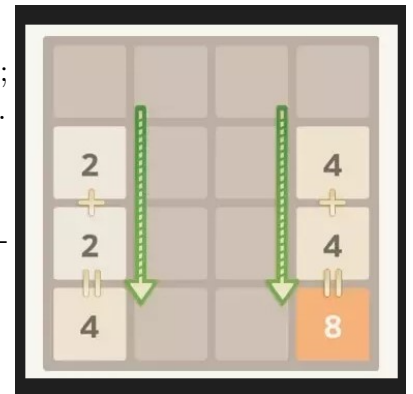
1. `public Game2048 ( )` Default constructor. It creates an empty 3×3 board.
2. `public Game2048 ( int size )` Creates an squared, empty, matrix $size \times size$. If the argument is incorrect, throws an `IllegalArgumentException` with the message "Invalid size".
3. `Game2048 ( int [ ] [ ] arg )` **Package protected**. It receives a board, probably empty, to be used as starting board. If it is null or has an invalid size or is not squared, or contains a value not power of two, default board is used instead. Very useful for testing purposes.

3.2.2 Methods for moving cells

The four movements must be implemented, with the same behaviour than in the original game.

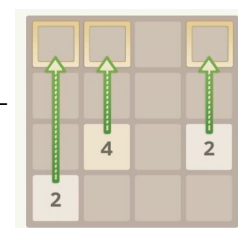
1. `public void moveRight ( )` slides and merges all cells to the right considering:

- Starting on front cell (right edge), if content is zero, ignore it; otherwise, slide it as far as possible to right, skipping zeroes.
- When a non-zero cell is found:
 - (a) If cell found equals to that of the cell moving, the movement is stopped, double the cell found (merge).
 - (b) If the value is different, the movement is stopped.

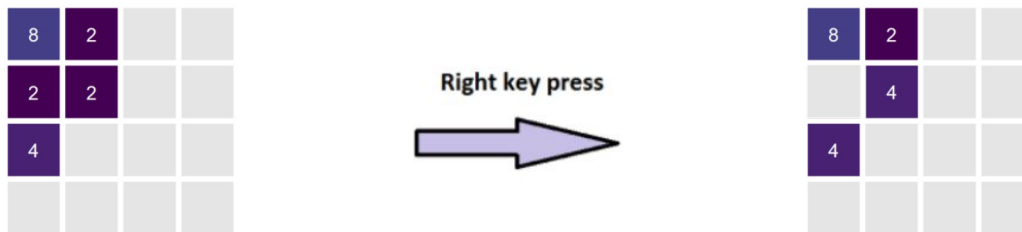


You can break this step into two tasks:

Make them slide: Cells slide in the direction of the movement, such that they are now stacked up in that direction.

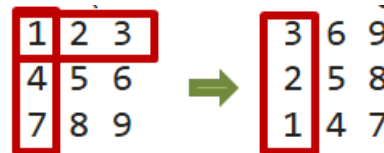


Make them merge: Adjacent cells are merged together.



2. `public void moveLeft ()` Slide and merge all numbers to the left. To carry out this operation, use the movement to the right together with left rotations of the board. This way, moving left could be implemented as `rotateLeft`, `rotateLeft`, move to the right, `rotateLeft`, `rotateLeft`.

The board must be rotated to the left (counter-clockwise) by using a method `rotateLeft`.



3. `public void moveUp ()` slides all numbers up. Similar to the previous one.
4. `public void moveDown ()` slides all numbers down. Similar to the previous one.

3.2.3 Other methods

`public boolean next ()` This method adds a new number 2 randomly on an empty cell. It will return true if a new number 2 could be added; false otherwise.

`public boolean isBoardFull ()` Check if the board is full. This occurs when all the positions on the board have a non-zero number.

`public String toString()` Return a string representation of the board. Give each cell 5 spaces (`String.format(...)`) and insert a new line after each row.

`int[] [] getBoard()` **Package protected.** Return the board.

`public boolean win()` Return true if the player has won; a tile contains 2048.

4 Game2048: Laboratory 01 and Laboratory 02

JUnit tests must be written for every public method and constructor. Always write tests first. Then write method body.

Write simple tests. **Test just one thing each time.** And divide your tests into three sections for **preparation, execution, and verification.**

Test class for method `isBoardFull` is **fully implemented.**

The goal of this first session is to **write tests and method body** for all constructors as well as methods `next` and `toString`.

To implement some tests it could be useful to use `assertArrayEquals`. **Notice the checklist provided.**

The goal of next session will be to finish all the test classes and methods.



Class `ForTesting` contains some useful code for implementing tests.

Write tests is mandatory for public methods. However, non public methods may also be tested (as `Game2048(int[] arg)`).

5 Documentation

In addition, you always have to generate the Javadoc documentation. The source code must contain both documentation and implementation comments (when needed for clarifying some complex algorithm).

Javadoc is needed for every public method and constructor. Use annotations (`@param`, `@return`, etc) for referring to parameters, returning value, etc.

The code provided is already documented with Javadoc and some other comments.

In addition, each test scenario must be fully described following the schema:

```
/**
 * GIVEN: preconditions to execute the method we are testing
 * WHEN: call the tested method
 * THEN: assertions on any results from the execution step are implemented.
 */
```

6 Check list

As no feedback will be provided from teachers, a checklist is available so you can check the correctness of your project according to the expected learning outcomes.

For project game, the same checklist is used for both, lab01 and lab02 as the implementation is splitted in two weeks.